

EXPERIMENT – 7

Performing Code Injection on Application Process Using Ptrace in Kali Linux

Aim:

To demonstrate code injection on a running application process using the ptrace system call in Kali Linux for understanding low-level process manipulation and security vulnerabilities.

Software Requirements:

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Programming Language:** C
- **Debugger / System Tool:** Ptrace
- **Compiler:** GCC
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Procedure:

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Create a Target Program

1. Open Terminal.
2. Create a simple C program:
nano target.c
3. Write a basic infinite loop program.

```
#include <stdio.h>

int main() {
    while(1) {
        printf("This is an infinite loop...\n");
    }
    return 0;
}
```

4. Compile the program:
5. gcc target.c -o target

Step 3: Run the Target Application

1. Execute the program:
./target
2. Note the Process ID (PID) using:
ps aux | grep target

Step 4: Create Injector Program

1. Create injector source file:
nano injector.c
2. Write ptrace-based code to attach to the target process and modify its memory or registers.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ptrace.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/user.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <pid>\n", argv[0]);
        return 1;
    }
    pid_t pid = atoi(argv[1]);
    struct user_regs_struct regs;
    if (ptrace(PTRACE_ATTACH, pid, NULL, NULL) == -1) {
        perror("ptrace attach failed");
        return 1;
    }
    waitpid(pid, NULL, 0);
    printf("Attached to process %d\n", pid);
    if (ptrace(PTRACE_GETREGS, pid, NULL, &regs) == -1) {
        perror("getregs failed");
        return 1;
    }
    printf("Original RIP: %llx\n", regs.rip);

    regs.rip += 2;

    if (ptrace(PTRACE_SETREGS, pid, NULL, &regs) == -1) {
        perror("setregs failed");
        return 1;
    }
    printf("Modified RIP: %llx\n", regs.rip);
    ptrace(PTRACE_DETACH, pid, NULL, NULL);
    printf("Detached from process\n");
    return 0;
}
```

3. Save the file.

Step 5: Compile the Injector Program

1. Compile the injector:
`gcc injector.c -o injector`

Step 6: Attach to Target Process

1. Attach injector to target process using PID:
`sudo ./injector <PID>`
2. Injector gains control of the target process.

Step 7: Inject and Execute Code

1. Modify target process memory or registers.
2. Resume execution of the target process.
3. Observe injected behavior.

Output:

```
(root@kali)-[~]
# nano target.c

(root@kali)-[~]
# gcc target.c -o target

(root@kali)-[~]
# ./target
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...

(root@kali)-[~]
# ps aux | grep ./target
root      2336  0.0  0.0   2568   1432 pts/0    S+   14:41   0:00 ./target
root      2500  0.0  0.0   6676   2288 pts/1    S+   14:41   0:00 grep --col
or=auto ./target

(root@kali)-[~]
# nano injector.c

(root@kali)-[~]
# gcc injector.c -o injector

(root@kali)-[~]
# sudo ./injector 2336
Attached to process 2336
Original RIP: 7fb8780d5687
Modified RIP: 7fb8780d5689
Detached from process

(root@kali)-[~]
#
```

Result:

Code injection using the ptrace system call was successfully demonstrated in Kali Linux. The experiment illustrates how process memory and execution flow can be manipulated, emphasizing the importance of secure process isolation.